

Python for data science

Nicolas Vannson, PhD

TBS - 2026

A bit of presentation?

Nicolas Vannson, PhD

Audiologist @ Brussels in 2010

Master in healthcare @ Montpellier in 2012

PhD in auditory neuroscience @ Toulouse in 2016

Yesterday :

Audiologist / CNRS associate researcher/ lecturer

Today :

Hearing Data Scientist – founder of Hearing Data Science



Hearing Data Science

An example in the industry

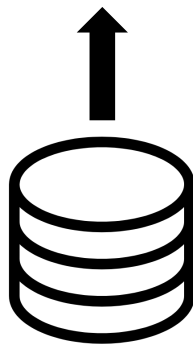
<https://hearingdatascience.com>



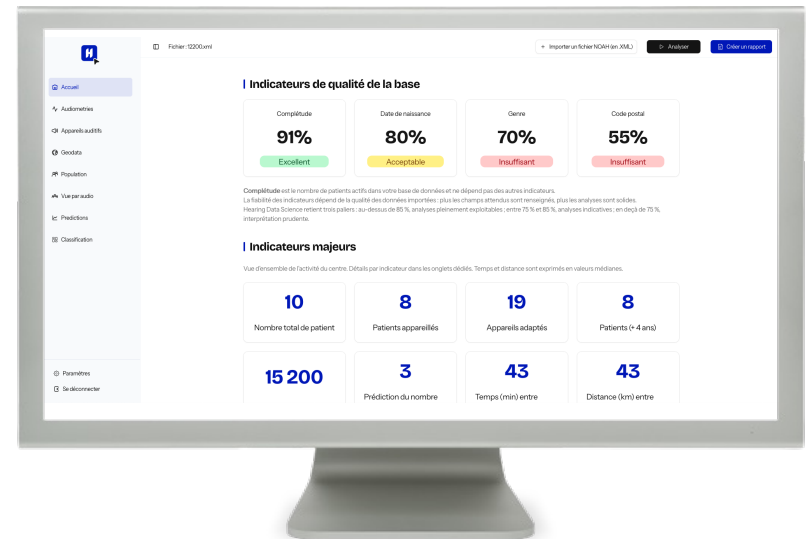
Hearing Data Science in details



GDPR cloud



Hearing center data base



Courses menu and evaluation?

Courses :

Python for data science

Advanced python for data science (December)

Evaluations :

Python for data science - personal project

Advanced python for data science – group project

Overall :

Questions (MCQ) for your final year project

Objectives?

1. Master the basics of data analysis
2. Create data visualisation & manipulating DataFrame
3. Write your own function

WHY PYTHON?



Why Python?



[About us](#) [Knowledge](#) [News](#) [Coding Standards](#) [TIOBE Index](#) [Contact](#) [🔍](#)

[Products](#) ▾

[Quality Models](#) ▾

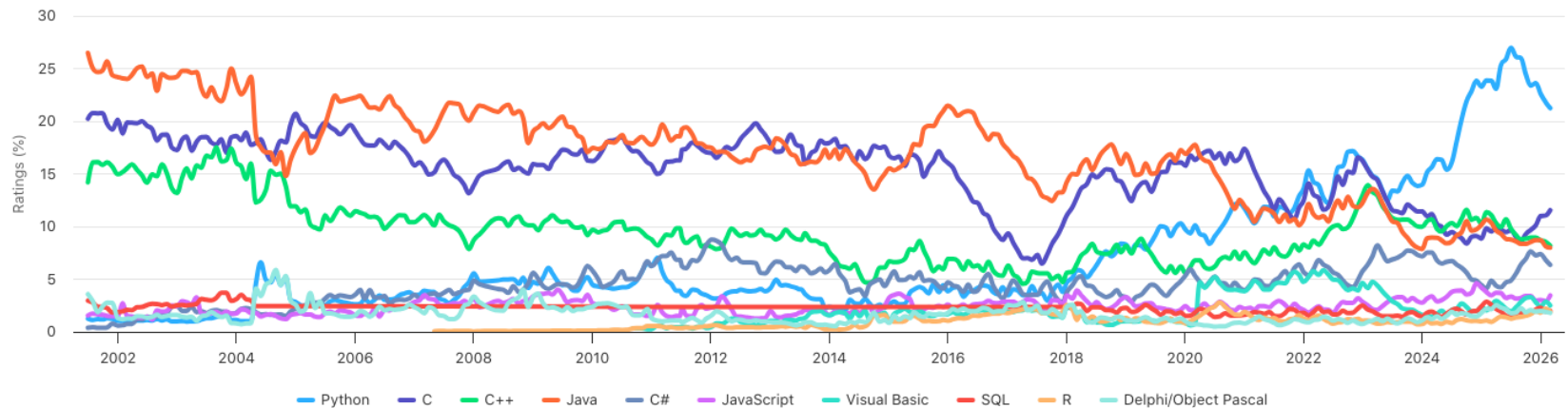
[Markets](#) ▾

[Schedule a demo](#)

18	13	⬇️	 PHP	1.23%	-0.25%
19	18	⬇️	 Ada	1.10%	+0.25%
20	26	⬆️	 Swift	1.04%	+0.44%

TIOBE Programming Community Index

Source: www.tiobe.com



Other programming languages

Python vs R?

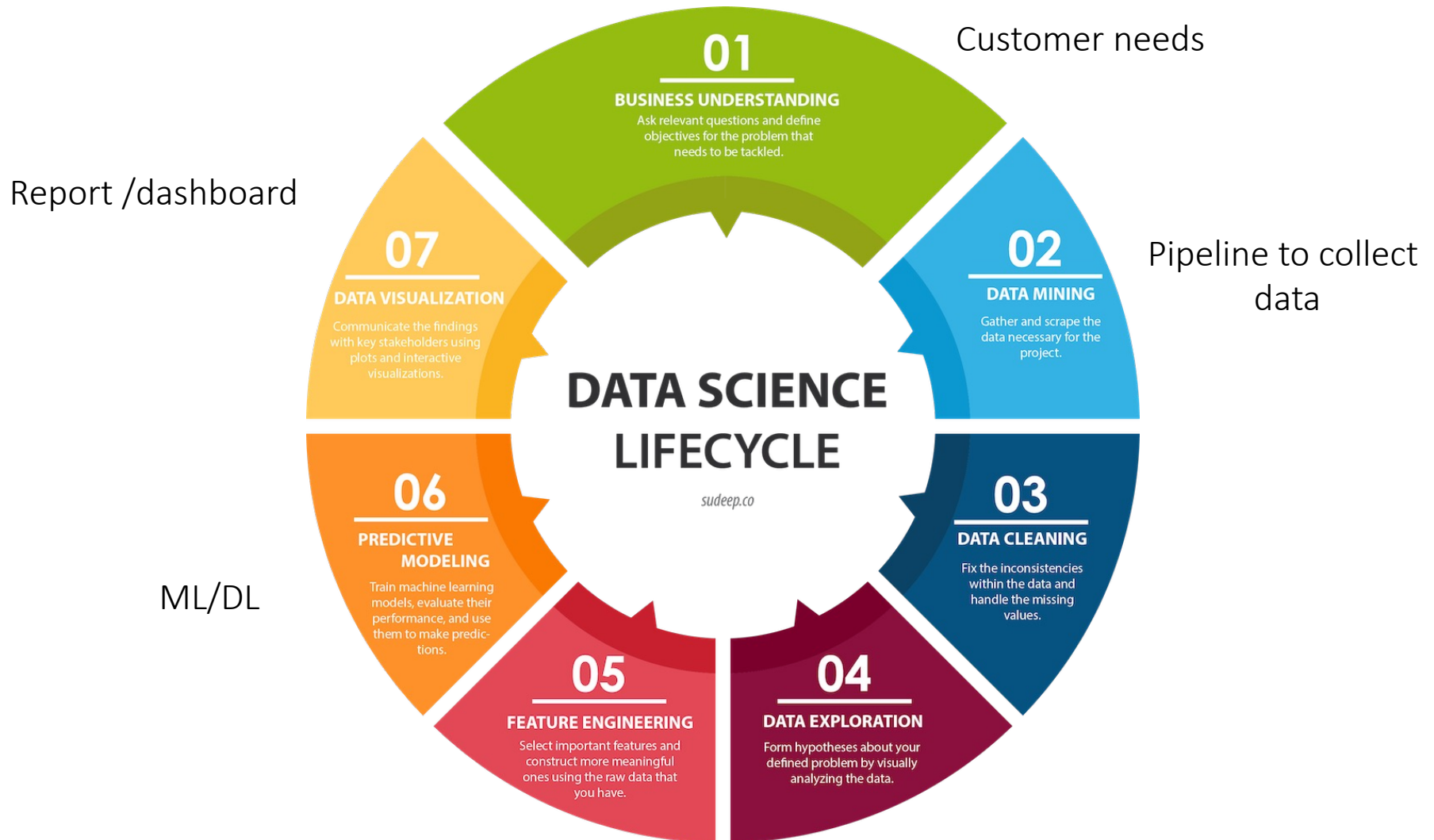
	Python	R
Philosophy	Focuses on readability & versatility.	Statistical. Built specifically for data analysis.
Learning Curve	Easy & Intuitive. Perfect for beginners.	Steeper. Requires math or statistical background.
Data Analysis	Powerful with Pandas & NumPy.	Outstanding via DataFrames & Tidyverse.
Data Visualization	Great (Seaborn, Matplotlib). Needs more code.	Industry standard with ggplot2 (stunning plots).
Machine Learning & AI	Absolute leader (Scikit-Learn, PyTorch, TensorFlow).	Great for traditional stats, weak for Deep Learning.
Production	Production-ready. Integrates with apps & web..	Research-focused. Harder to deploy in software.

“90 %”

Guillame Saint-Cirgue, machine learnia founder

WHY DATA SCIENCE ?

Data science a whole process?

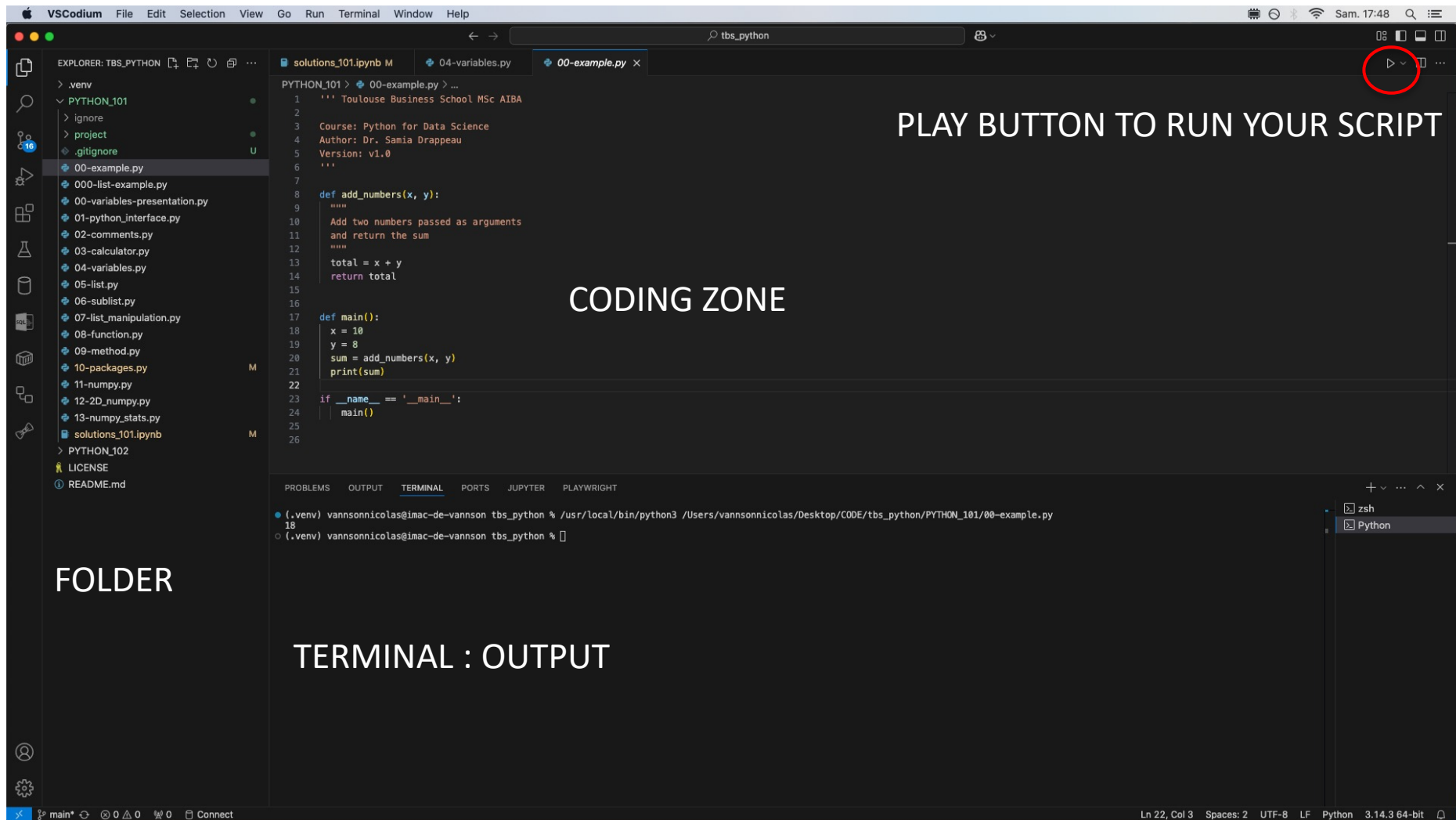


- This course is an hand-on course
- 3 exercises and correction
- 15-20 minutes for all exercises
- Go as your own pace !
- Ask a student to answer to each exercise

LET'S PRACTICE



VS Code the IDE



A python script!

- A python script ends with `.py` (extension) => terminal is another window
- A python notebook ends with `.ipynb` (advanced python course)
terminal below code block

```
01-python_interface.py

# This file is a Python script
# Experiment with it to get familiar with the Python Interface

msg = "Hello, TBS!"
print(msg)

compute = 5/208
print(compute)

[1]

... Hello, TBS!
0.02403846153846154
```

Clone the GitHub repo?

Cloning a Git repository:

- Copy the repository URL: https://github.com/MEMAUDATA/tbs_python.git
- In VS Code Welcome window, click on “Clone Git Repository”.
- Paste the URL into the input field at the top and select Desktop as the destination folder.
- Click on Open (or Open Repository) when prompted.
- In the File Explorer, navigate to the [PYTHON_101](#) folder to access the exercises.

- 01-python_interface.py
- 02-comments.py
- 03-calculator.py

LET'S PRACTICE



Variables and types?

- A variable is a named storage for a value that can change throughout the code

```
a = 2
print(a**2)
>>> 4
```

- Type

- Numbers : float (3.14) or int (3)
- Text : str ("2" or '2')
- Boolean : bool (True or False)

- Different types cause different behaviour

```
2      + 2      >>> 4 (=> int)
"2"    + "2"    >>> "22" (=> str)
```

- Always check the variable's type to avoid simple mistakes !

```
price = 100
type(price)
>>> int
```

List?

- A list is a named collection of values or variables
- Can contain any type (even a list)
- Example :

```
savings = [100,200,300,400,500]
type(savings)
>>> list
client_savings = ["c1", 100, "c2", 200]
type(client_savings)
>>> list
```

Get access to an item of the list?

- Use **index** to get access to a element of the list. First element starts at 0 (not 1) and to get access to the last element : -1

```
savings = [100, 200, 300, 400, 500]
print(savings[0])
>>> 100
print(savings[-1])
>>> 500
```

- List slicing : [start: end] => include start and exclude end

```
print(savings[1:3])
>>> [100,200]
print(savings[:3])
>>> [100,200]
print(savings[1:])
>>> [200,300,400,500]
```

List manipulation?

- Manipulate one or more values within the list (change, add, remove)

```
savings = [100,200,300,400,500]
savings[0] = 800
print(savings)
>>> [800,200,300,400,500]
savings[1:3] = [8000,1000]
print(savings)
>>> [800,8000,1000,400,500]
```

```
bigger_savings = savings + [100]
print(bigger_savings)
>>> [800,8000,1000,400,500,100]
```

```
del bigger_savings[0]
print(bigger_savings)
>>> [8000,1000,400,500,100]
```

- 04-variables.py
- 05-list.py
- 06-sublist.py
- 07-list_manipulation.py

LET'S PRACTICE



Function?

- A function is a reusable block of code

```
# Custom function
```

```
def add_numbers(a,b):
```

```
    print(a+b)
```

```
add_numbers(1,3)
```

```
# built-in function
```

```
min(savings)
```

```
max(savings)
```

```
# need help on the function
```

```
help(min)
```

Method?

- A method (`method()`) is a function linked (by a `.`) to an object

```
savings = [100, 200, 300, 400, 500]
# Add 100 to the list of savings
savings.append(100) # .append() is a method
```

```
# Count the number of times 100 appears
>>> [100, 200, 300, 400, 500, 100]
savings.count(100) #.count is also a method
>>> 2
```

```
name = "liz"
name.capitalize()
>>> "Liz"
name.replace("z", "sa")
>>> "Lisa"
```

- 08-function.py
- 09-method.py

LET'S PRACTICE



What's a python package?

- Package is a directory of Python scripts
- Each script of a package is called a module
- Modules specify functions, methods and new Python types aimed at solving particular problems
- Packages for data science:
 - NumPy to efficiently work with arrays
 - Pandas for data analysis
 - Matplotlib for data visualization
 - scikit-learn for machine learning
- Not all these packages are available in Python by default

Install and use a python package?

- Install a package
- Using a command in the terminal : `pip` (sometimes `uv`)
`pip install numpy`
- Use a package
- In a script, at the top of it, you have to import the package in order to use it

```
import numpy
import numpy as np # better way to import package
# Compute mean
students_mean = np.mean(students)
# Compute mean without as np
students_mean = numpy.mean(students)
```

- Use a module from a package

```
from numpy import random
random.randint(1,10)
```

NumPy?

- Really important for data science (such as pandas as well)
- Numpy : Numerical Python
- List of numbers called numpy arrays (better for data manipulation than basic python list)

```
import numpy as np
height      = [1.85, 1.79, 1.67]
np_height   = np.array(height)
weight      = [65.4, 59.2, 63.6]
np_weight   = np.array(weight)
bmi         = np_weight/np_height**2
print(bmi)
>>> [19.10883857 18.47632721 22.80469002]
```

```
weight/height**2 # using basic python list
>>> TypeError: unsupported operand type(s) for ** or pow():
'list' and 'int'
```

NumPy?

- Filter an array ?

```
print(bmi)
>>> [19.10883857 18.47632721 22.80469002]
print(bmi>20)# is there any bmi values above 20 ?
>>> [False False True]
```

```
upper_20 = bmi>20 # give me all the bmi value above 20
print(bmi[upper_20])
>>> [22.80469002]
```

```
print(bmi[bmi>20])
>>> [22.80469002]
```

NumPy?

- Filter an array ?

```
print(bmi)
>>> [19.10883857 18.47632721 22.80469002]
print(bmi>20)# Are there any BMI values above 20?
>>> [False False True]
```

```
upper_20 = bmi>20 # Give me all the BMI values above 20
print(bmi[upper_20])
>>> [22.80469002]
```

```
print(bmi[bmi>20])
>>> [22.80469002]
```

- 10-packages.py
- 11-numpy.py

LET'S PRACTICE



2D-NumPy?

- A 2D NumPy array means that an array can have multiple rows and columns

```
np_height = np.array([1.85, 1.79, 1.67])  
np_weight = np.array([65.4, 59.2, 63.6])  
type(np_height)  
>>> numpy.ndarray # same type for np_weight
```

```
np_2d = np.array([[1.85, 1.79, 1.67], [65.4, 59.2, 63.6]])  
np_2d.shape  
>>>(2,3) # 2 rows, 3 columns
```

2D-NumPy?

- Selecting elements within np_2d

```
np_2d = np.array([[1.85, 1.79, 1.67],  
                  [65.4, 59.2, 63.6]])
```

```
np_2d[0]
```

```
>>> array([1.85, 1.79, 1.67])
```

```
np_2d[:, 1:3] # select all rows and columns 1 and 2
```

```
>>> array([[ 1.79, 1.67], [59.2 , 63.6 ]])
```

Basic NumPy stats?

```
import numpy as np
np_2d = np.array([[1.85, 1.79, 1.67],
                  [65.4, 59.2, 63.6]])
np.mean(np_2d[:, 0]) # mean of the first column
>>> 33.625
```

```
np.std(np_2d[:, 0]) # std of the first column
>>> 31.775
```

there's a ton of stats you can do with NumPy ... Let's practice!

- 12-2D_numpy.py
- 13-numpy_stats.py

LET'S PRACTICE



END COURSE EVALUATION



End course evaluation

- In PYTHON_101 folder lies the project folder containing the script template with all the instructions
- Rename the template with your name and PYTHON_101,
ex: VANNSON_PYTHON_101.py
- When done, load it on campus
- You'll have a week, deadline : 23/10/2026 @23h59



vannson.nicolas@gmail.com